

# Geobot / Agente Educativo — Documentación de entrega

---

## Geobot / Agente Educativo — Documentación de entrega (handoff)

---

Documento para quien asuma el mantenimiento del proyecto. Explica **qué es, dónde está cada cosa, cómo se conectan frontend y backend, cómo se implementa cada LLM**, y la **propuesta del agente inteligente** (varios modelos en uno, según la complejidad de la pregunta).

---

### 1. Qué es este proyecto (en una frase)

---

Un MVP de **tutor sobre documentos locales** (PDF, DOCX, XLSX, MD, TXT): el usuario elige una biblioteca y un archivo, hace preguntas, y el sistema responde basándose en el contenido del documento (más extractos de otros archivos del mismo alcance).

No es un agente con tools / function calling. Es un **RAG-lite**:

1. Lee el archivo activo desde disco.
2. Busca por palabras en otros documentos (sin embeddings).
3. Arma un system prompt de "tutor".
4. Envía todo a un LLM (Gemini, Groq, DeepSeek, etc.).

La idea central del producto es el **Agente inteligente (Smart)**: un solo chat que **elige automáticamente** entre Gemini / Groq / DeepSeek según la pregunta y el tamaño del documento.

---

## 2. Stack y cómo arrancar

| Capa       | Tecnología                                           |
|------------|------------------------------------------------------|
| Frontend   | Next.js 14 (App Router) + React 18 + TypeScript      |
| Backend    | Node.js + Express 4 + TypeScript (ESM)               |
| Documentos | Carpeta local en disco ( backend/data/ por defecto)  |
| LLMs       | Gemini, Groq, DeepSeek, OpenAI, Hugging Face, Ollama |

### Qué viene en el clone y qué NO

| Ítem                            | ¿Viene al clonar?          | Acción                                                                                       |
|---------------------------------|----------------------------|----------------------------------------------------------------------------------------------|
| Código frontend/ + backend/src/ | Sí                         | Nada extra                                                                                   |
| frontend/.env.local             | Sí (está trackeado en Git) | Ya apunta a http://localhost:3001/api/chat . Solo cámbialo si el backend usa otra URL/puerto |
| backend/.env.example            | Sí                         | Es la plantilla sin keys                                                                     |
| backend/.env                    | No (gitignore)             | Hay que crearlo / recibirlo (ver abajo)                                                      |
| backend/data/                   | No (gitignore)             | Hay que copiarla desde OneDrive/ZIP (ver abajo)                                              |

Sin .env no hay LLM. Sin data/ el wizard no muestra bibliotecas.

### Primera vez: preparar .env y la carpeta data/ (obligatorio)

Haz esto antes de npm run dev .

#### A) Backend .env (API keys + puerto)

1. Entra a la carpeta del backend:

```
cd backend
```

2. Crea el archivo de secrets a partir de la plantilla o usa el .env que te entregaron en el handoff:

```
# Opción 1 – te pasaron el .env listo (recomendado en traspaso):
# Copia el archivo recibido a: backend/.env

# Opción 2 – crearlo desde cero:
cp .env.example .env
```

3. Abre `backend/.env` y asegúrate de tener **como mínimo**:

```
PORT=3001
GEOTRENDS_DOCUMENTS_ROOT=data

GEMINI_API_KEY=...tu_key...
GROQ_API_KEY=...tu_key...
DEEPSEEK_API_KEY=...tu_key...
```

- `PORT=3001` es obligatorio para alinear con el front (si falta, el código usa 4000 y el front no conecta).
- `GEOTRENDS_DOCUMENTS_ROOT=data` apunta a la carpeta de documentos relativa al backend.
- Las keys de OpenAI / Hugging Face / Ollama solo hacen falta si vas a probar esos paneles.

**Nunca** subas `backend/.env` a Git. Contiene secretos.

## B) Carpeta `backend/data/` (documentos / bibliotecas)

Esta carpeta **no está en el repo**. La persona saliente debe dejarla en OneDrive (ZIP). Tú debes montarla así:

1. Descarga el ZIP desde OneDrive (ej. `geotrends-documentos-data.zip`).
2. Descomprímelo de modo que la ruta final quede **exactamente**:

```
agentePrueba/backend/data/
├─ Ficha de Servicios/
├─ Guía generica/
└─ Manuales/
```

3. Comprobación rápida:

```
ls backend/data
# Debe listar las bibliotecas (carpetas), no un "data" anidado raro tipo
  backend/data/data/...
```

Si quedó `backend/data/data/...` por un error al descomprimir, muévelo un nivel arriba.

Variables relacionadas: - El backend lee `GEOTRENDS_DOCUMENTS_ROOT` (valor típico: `data`). - Cada **subcarpeta de primer nivel** dentro de `data/` aparece como **Biblioteca** en la UI.

---

## Arranque local para probar (después de A y B)

El frontend espera el backend en `http://localhost:3001`.

### 1) Backend

```
cd backend
npm install
# backend/.env y backend/data/ ya deben existir (pasos A y B)
npm run dev
```

Debe imprimir algo como: `Agent backend running on http://localhost:3001`.

Comprueba: `GET http://localhost:3001/health` → `{ "ok": true }`.

## 2) Frontend (en otra terminal)

```
cd frontend
npm install
# .env.local ya viene en el repo; no hace falta crear uno si usas puerto 3001
npm run dev
```

UI: `http://localhost:3000`.

## 3) Probar el flujo

1. Elige una biblioteca → carpeta/archivo → documento.
2. Debe verse el preview y el **Agente inteligente**.
3. Pregunta corta (ej. "¿de qué trata?") → suele ir a **Groq**.
4. Pregunta analítica (ej. "explica en detalle...") → **DeepSeek** o Gemini.
5. Mira en la UI el badge del modelo y el motivo de routing.

**Importante — puerto:** si `backend/.env` no define `PORT`, el código cae a **4000** (`index.ts`). El front espera **3001**. Siempre define `PORT=3001` en `.env`.

**Fuente de verdad:** este documento + el PDF homónimo. El `README.md` es un resumen corto.

---

## 3. Estructura del repositorio

```
agentePrueba/
├── README.md           # Resumen + enlace a este handoff
├── DOCUMENTACION-HANDOFF.md # ← este documento (markdown)
├── DOCUMENTACION-HANDOFF.pdf # Misma documentación en PDF
├── frontend/          # UI Next.js (Geobot)
└── backend/           # API Express + lógica LLM
```

Son **dos apps independientes** (sin monorepo npm workspaces ni Docker). Cada una tiene su propio `package.json`, `node_modules` y `.env`.

**Repo remoto (GitHub):** <https://github.com/isabelasring/agente.git> (rama `main`).

---

## 4. Backend — qué hay en cada carpeta

---

```
backend/
├── package.json           # Scripts: dev (tsx watch), build (tsc), start
├── tsconfig.json          # strict, module NodeNext, outDir dist/
├── .env / .env.example    # Keys y PORT
├── data/                  # Corpus documental (bibliotecas)
│   ├── Ficha de Servicios/
│   ├── Guia generica/
│   └── Manuales/
├── src/
│   ├── index.ts           # Entry: Express, CORS, /health, monta /api
│   ├── routes/
│   │   └── chat.ts        # TODAS las rutas HTTP del producto
│   ├── services/
│   │   ├── context.ts     # FS: listar, leer, parsear, cache, path-safe
│   │   ├── search.ts      # Búsqueda lexical multi-documento
│   │   ├── llm.ts         # Orquestador: askTutor + askSmartTutor
│   │   ├── providerRouter.ts # ★ Propuesta smart: elige Gemini/Groq/DeepSeek
│   │   └── contextBudget.ts # Truncado de contexto por proveedor
│   ├── prompts/
│   │   └── tutorPrompt.ts # System prompt del tutor (rol, fuentes, tono)
│   ├── shared/
│   │   └── chatHistory.ts # Historial: normalizar, trim (máx 12 msgs), formato
├── OpenAI
│   ├── llmTypes.ts        # AskProviderInput + helpers de error
│   ├── gemini/askGemini.ts # Adapter Gemini (SDK oficial)
│   ├── groq/askGroq.ts    # Adapter Groq (fetch OpenAI-compatible + shrink 413)
│   ├── deepseek/askDeepSeek.ts # Adapter DeepSeek (fetch OpenAI-compatible)
│   ├── openai/askOpenAI.ts # Adapter OpenAI
│   ├── huggingface/askHuggingFace.ts
│   └── ollama/askOllama.ts # Adapter Ollama local
```

### 4.1 Entry point — `src/index.ts`

- Carga `dotenv/config`.
- `cors()` abierto + `express.json()`.
- `GET /health`.
- Monta el router en `/api` → todo lo de `routes/chat.ts` queda bajo `/api/...`.
- Puerto: `process.env.PORT || 4000` (en `.env.example` está `3001`).

## 4.2 Rutas — `src/routes/chat.ts` (contrato HTTP)

| Método | Ruta                                                   | Para qué                                |
|--------|--------------------------------------------------------|-----------------------------------------|
| POST   | <code>/api/chat</code>                                 | Pregunta al tutor (fijo o auto-route)   |
| GET    | <code>/api/documents?<br/>courseId=&amp;folder=</code> | Lista documentos indexables             |
| GET    | <code>/api/folders</code>                              | Bibliotecas top-level                   |
| GET    | <code>/api/folders?parent=</code>                      | Hijos (dirs + files) de una biblioteca  |
| GET    | <code>/api/document?path=</code>                       | Binario (PDF iframe, DOCX, etc.)        |
| GET    | <code>/api/document/text?<br/>path=</code>             | Texto extraído (no PDF; máx ~80k chars) |
| GET    | <code>/api/document/html?<br/>path=</code>             | HTML tablas XLSX                        |

### Body de POST `/api/chat`

```
{
  "message": "¿De qué trata este documento?",
  "courseId": "geotrends",
  "documentId": "Manuales/algun-pdf.pdf",
  "folder": "Manuales",
  "provider": "auto",
  "autoRoute": true,
  "history": [
    { "role": "user", "content": "..."},
    { "role": "assistant", "content": "..."}
  ]
}
```

| Campo                    | Obligatorio | Notas                                                                                        |
|--------------------------|-------------|----------------------------------------------------------------------------------------------|
| message                  | Sí          | Pregunta del usuario                                                                         |
| courseId                 | Sí          | Valida el body; hoy el FS <b>no filtra por courseId</b> (legacy).<br>Valor típico: geotrends |
| documentId (o lessonId ) | Sí          | Ruta relativa al root documental, o sentinel<br>__geotrends_no_doc__                         |
| folder                   | No          | Prefijo biblioteca/subcarpeta para acotar inventario y búsqueda                              |
| provider                 | No          | gemini   groq   deepseek   openai   huggingface   ollama   auto                              |
| autoRoute                | No          | Si true o provider === "auto" → entra al smart router                                        |
| history                  | No          | Solo el Smart lo envía hoy; el backend lo recorta a 12 mensajes                              |

Respuesta OK

```
{
  "answer": "texto markdown de la respuesta...",
  "meta": {
    "courseId": "geotrends",
    "documentId": "...",
    "provider": "groq",
    "providerLabel": "Groq",
    "autoRoute": true,
    "routingReason": "Pregunta directa o corta → Groq (rapido).",
    "historyMessages": 2,
    "folder": "Manuales",
    "contextLoaded": true,
    "contextChars": 12345,
    "inventorySize": 40,
    "searchHits": [{ "path": "...", "score": 3 }]
  }
}
```

routingReason y providerLabel son lo que muestra el panel Smart en la UI.

4.3 Flujo interno de un chat (paso a paso)

Cuando llega POST /api/chat :

1. Valida message + courseId + documentId
2. listCourseDocuments(folder) → inventario
3. Resuelve documentId (si no está en inventario, usa el primero)
4. En paralelo:
  - loadCourseContext → texto completo del documento activo
  - searchAcrossDocuments → hasta 3 snippets de otros archivos
5. Si autoRoute → askSmartTutor(...)  
Si no → askTutor(..., provider)
6. Responde { answer, meta }

#### 4.4 Documentos — services/context.ts

- Root: GEOTRENDS\_DOCUMENTS\_ROOT (relativa al cwd del backend o absoluta). Default conceptual: data .
- Cada **subcarpeta directa** de data/ es una **"Biblioteca"** en la UI ( Ficha de Servicios , Guía generica , Manuales ).
- Extensiones indexables: .md , .txt , .docx , .pdf , .xlsx .
- Parseo: PDF ( pdf-parse ), DOCX ( mammoth ), XLSX ( xlsx ), texto plano.
- Cache en memoria por mtimeMs (si el archivo no cambia, no se re-parsea).
- safeResolveUnderRoot : evita path traversal ( ../ fuera del root).
- Sentinel alineado con el front: \_\_geotrends\_no\_doc\_\_ .

#### 4.5 Búsqueda — services/search.ts

Búsqueda **lexical** (palabras clave), no vectorial:

- Tokeniza la pregunta, quita stopwords ES/EN.
- Score por frecuencia en el texto + bonus si aparece en el nombre del archivo.
- Devuelve snippets; el prompt los mete como "EXTRACTOS DE OTROS DOCUMENTOS".

#### 4.6 Prompt del tutor — prompts/tutorPrompt.ts

buildSystemPrompt(...) arma el system message con:

- Rol: analizar docs técnicos/normas sin inventar.
- Orden de fuentes: documento activo → extractos → inventario (solo existencia).
- Instrucciones de formato markdown y tono afirmativo.

Todos los adapters LLM pasan por este prompt.

#### 4.7 Orquestador LLM — services/llm.ts

Dos funciones clave:

| Función                    | Uso                                                                                    |
|----------------------------|----------------------------------------------------------------------------------------|
| <code>askTutor</code>      | Panel fijo: llama al adapter según <code>provider</code>                               |
| <code>askSmartTutor</code> | Agente inteligente: <code>pickProvider</code> → budget → ask → posible fallback Gemini |

`askTutor` hace un `switch` a:

- `askGemini`, `askGroq`, `askDeepSeek`, `askOpenAI`, `askHuggingFace`, `askOllama`.

Si no hay contexto ni snippets, devuelve un mensaje claro de "no hay contenido cargado...".

---

## 5. La propuesta: un agente con varios LLM según complejidad

### 5.1 Idea de producto

En lugar de forzar al usuario a elegir Gemini vs Groq vs DeepSeek:

1. Un solo chat (**Agente inteligente**).
2. El backend **clasifica** la pregunta (y mira tamaño del doc / historial).
3. Enruta a:
  - **Groq** → rápido, barato, preguntas cortas/directas.
  - **DeepSeek** → analítico, detallado, seguimiento / complejidad.
  - **Gemini** → documentos enormes, default general, y **fallback** si Groq/DeepSeek fallan.

Los otros paneles (OpenAI, HF, Ollama, etc.) existen para **comparar proveedores lado a lado** en demos; el producto "oficial" es el Smart.

### 5.2 Dónde vive la lógica de decisión

Archivo: `backend/src/services/providerRouter.ts` → función `pickProvider`.

También usa `contextBudget.ts` → `isComplexQuestion` (regex de complejidad + historial largo ≥ 10 mensajes).

Heurísticas de pickProvider (orden aproximado)

| Condición                                                                              | Proveedor | Motivo típico      |
|----------------------------------------------------------------------------------------|-----------|--------------------|
| Pregunta “simple” (regex: título, qué es, fecha...) o corta (< 75 chars) y no compleja | Groq      | Rápido             |
| Pregunta compleja y hay DEEPSEEK_API_KEY                                               | DeepSeek  | Analítico          |
| Compleja sin DeepSeek                                                                  | Gemini    | Fallback analítico |
| Documento > 130 000 caracteres                                                         | Gemini    | Contexto grande    |
| Pregunta < 130 chars y poco historial                                                  | Groq      | Breve              |
| Pregunta ≥ 130 chars y DeepSeek OK                                                     | DeepSeek  | Detallada          |
| Resto                                                                                  | Gemini    | General            |

Regex de “simple” (extracto): titulo|nombre|de que trata|resumen|objeto|cual es|que es|fecha|version|...

Regex de “compleja” ( contextBudget.ts ): segun lo|compara|articulo|explica en detalle|diferencia entre|calcula|plazo|porque|detalla|fundamenta|requisito|procedimiento|paso a paso|... o historial ≥ 10.

5.3 Presupuesto de contexto por modelo — contextBudget.ts

Antes de llamar al LLM, se recorta el texto del documento:

| Proveedor | Límite aproximado                                          |
|-----------|------------------------------------------------------------|
| Gemini    | 180 000 chars                                              |
| DeepSeek  | 32 000 (normal) / 48 000 (compleja)                        |
| Groq      | 12 000–18 000 (más agresivo; Groq suele devolver HTTP 413) |

5.4 Fallback a Gemini — askSmartTutor

Si la respuesta de Groq/DeepSeek “parece error” (413, auth, cuota, DeepSeek no configurado), se reintenta con Gemini y se concatena en routingReason :

... Fallback → Gemini (groq no disponible o limite).

Eso se ve en la UI bajo “Modelo: ...”.

## 5.5 Flujo end-to-end del Agente inteligente



## 5.6 Frontend del Smart — frontend/smart/SmartAgentPanel.tsx

Particularidades vs paneles fijos:

- Envía provider: "auto" y autoRoute: true.
- Envía history (todos los mensajes excepto el welcome).
- El backend limita historial a **12 mensajes** (= hasta ~6 intercambios).
- Lee meta.providerLabel y meta.routingReason y los muestra.
- Shell visual distinto ( .smart-agent-shell , color smart).

Los paneles de comparación (GeminiPanel, GroqPanel, ...) envían un provider fijo y **no** mandan historial (cada turno es independiente hacia el API, aunque la UI muestre chat).

## 6. Cómo se implementa cada LLM (adapters)

Patrón común en todos:

1. Reciben AskProviderInput ( message , courseContext , inventario, snippets, history ).
2. Construyen system prompt con buildSystemPrompt .

3. Llamen a su API.

4. Devuelven un **string** (respuesta o mensaje de error legible). No lanzan siempre; varios errors se convierten en texto.

| Proveedor    | Archivo backend                   | Endpoint / SDK real                                          |
|--------------|-----------------------------------|--------------------------------------------------------------|
| Gemini       | src/gemini/askGemini.ts           | SDK @google/generative-ai                                    |
| Groq         | src/groq/askGroq.ts               | https://api.groq.com/openai/v1/chat/completions              |
| DeepSeek     | src/deepseek/askDeepSeek.ts       | https://api.deepseek.com/chat/completions                    |
| OpenAI       | src/openai/askOpenAI.ts           | https://api.openai.com/v1/chat/completions                   |
| Hugging Face | src/huggingface/askHuggingFace.ts | https://router.huggingface.co/v1/chat/completions            |
| Ollama       | src/ollama/askOllama.ts           | {OLLAMA_BASE_URL}/api/chat (default http://127.0.0.1:11434 ) |

### Dónde sacar cada API key

| Proveedor    | URL típica                                                                                                                        |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Gemini       | <a href="#">Google AI Studio</a>                                                                                                  |
| Groq         | <a href="https://console.groq.com/keys">console.groq.com/keys</a>                                                                 |
| DeepSeek     | <a href="https://platform.deepseek.com/api_keys">platform.deepseek.com/api_keys</a>                                               |
| OpenAI       | <a href="https://platform.openai.com/api-keys">platform.openai.com/api-keys</a>                                                   |
| Hugging Face | <a href="https://huggingface.co/settings/tokens">huggingface.co/settings/tokens</a> (permiso "Make calls to Inference Providers") |
| Ollama       | No usa cloud key; instalar Ollama y <code>ollama pull llama3.2</code>                                                             |

### Notas por proveedor

- **Gemini:** modelo por defecto `gemini-flash-latest` . Es el más tolerante a documentos grandes y el fallback del smart.
- **Groq:** si el payload es muy grande → HTTP 413. El adapter hace **retry con shrink progresivo** (recorta doc, inventario, snippets) varias veces. Por eso no conviene mandar PDFs enormes enteros a Groq.

- **DeepSeek:** API compatible con Chat Completions; model default en `.env.example : deepseek-v4-flash`. Si no hay key, el router cae a Gemini en preguntas complejas.
- **OpenAI / HF / Ollama:** disponibles en paneles de comparación; **no entran** al auto-route del Smart.

Panel frontend correspondiente (solo UI; la API real está en backend):

| UI           | Carpeta / archivo                                      |
|--------------|--------------------------------------------------------|
| Smart        | <code>frontend/smart/SmartAgentPanel.tsx</code>        |
| Gemini       | <code>frontend/gemini/GeminiPanel.tsx</code>           |
| Groq         | <code>frontend/groq/GroqPanel.tsx</code>               |
| DeepSeek     | <code>frontend/deepseek/DeepSeekPanel.tsx</code>       |
| OpenAI       | <code>frontend/openai/OpenAIPanel.tsx</code>           |
| Hugging Face | <code>frontend/huggingface/HuggingFacePanel.tsx</code> |
| Ollama       | <code>frontend/ollama/OllamaPanel.tsx</code>           |

## 7. Frontend — qué hay en cada carpeta

```

frontend/
├─ package.json
├─ next.config.mjs
├─ .env.local                # URL del API + courseId
├─ app/
│   ├─ layout.tsx           # Shell HTML + metadata + globals.css
│   ├─ page.tsx             # ★ Única página: wizard docs + paneles
│   └─ globals.css          # Estilos globales (~700 líneas)
├─ shared/                  # Código compartido entre paneles
│   ├─ agentConfig.ts       # URLs, COURSE_ID, sentinels, buildFolderPrefix
│   ├─ agentTypes.ts        # ChatMessage, DocumentItem, FolderEntry
│   ├─ useCourseDocuments.ts # Hooks: docs, carpetas top, hijos biblioteca
│   ├─ DocumentPreview.tsx  # Preview PDF / DOCX / texto / XLSX
│   ├─ MessageContent.tsx   # Assistant → markdown; user → texto
│   └─ ChatMarkdown.tsx     # Markdown casero (sin react-markdown)
├─ smart/SmartAgentPanel.tsx # Agente auto-route
├─ gemini|groq|deepseek|openai|huggingface|ollama/ # Paneles por proveedor
├─ public/agent-avatar.png
└─ agent/                   # Carpeta vacía (sin uso)

```

### 7.1 Página principal — `app/page.tsx`

Orquesta Geobot:

1. **Paso 1:** elegir biblioteca ( `useTopLevelFolders` → `GET /api/folders` ).
2. **Paso 2:** elegir carpeta o archivo dentro ( `useLibraryChildren` ).
3. **Paso 3:** si hizo falta, elegir documento concreto ( `useCourseDocuments` ).

4. **Preview** del documento ( `DocumentPreview` ).
5. Solo con selección lista:
  - Sección **Agente inteligente** ( `SmartAgentPanel` ).
  - Grid **Comparación por proveedor** (6 paneles).

Estado levantado en la página: biblioteca, subcarpeta, `selectedDocumentId` , etc.

Al cambiar documento/carpeta, los paneles se remontan con `key={...}` → **se reinicia el chat** (intencional).

## 7.2 Config de URLs — `shared/agentConfig.ts`

```
AGENT_CHAT_URL = NEXT_PUBLIC_AGENT_API_URL || "http://localhost:3001/api/chat"
```

El resto se **deriva** reemplazando `/chat` :

- `/documents` , `/folders` , `/document` , `/document/text` , `/document/html`

Constantes importantes:

- `COURSE_ID` ← `NEXT_PUBLIC_COURSE_ID` (default `geotrends` )
- `NO_DOCUMENT_SENTINEL` = `__geotrends_no_doc__`
- `buildFolderPrefix(library, subfolder)` → string que el backend usa como `folder`

## 7.3 Cómo se conecta front ↔ back (de punta a punta)

```
Browser (localhost:3000)
  | fetch JSON
  ▼
Next.js (solo sirve UI; no proxyea el LLM)
  | NEXT_PUBLIC_AGENT_API_URL
  ▼
Express (localhost:3001/api/...)
  |
  ├── /folders, /documents, /document* → context.ts + FS (data/)
  └── /chat                            → search + llm + adapters → APIs externas
                                      Gemini / Groq / DeepSeek / ...
```

Comunicación: **solo HTTP request/response** con `fetch` . No hay WebSockets ni SSE/streaming.

CORS: el backend tiene `cors()` sin whitelist → en local funciona sin más.

## 7.4 Preview de documentos

`DocumentPreview.tsx` :

- PDF → iframe a `GET /api/document?path=`
- DOCX → descarga binario + `docx-preview` en el browser
- MD/TXT → `GET /api/document/text`
- XLSX → `GET /api/document/html` (HTML de tablas; se inyecta en el DOM)

---

## 8. Variables de entorno (backend)

---

Plantilla: `backend/.env.example`

| Variable                                                                                    | Para qué                                                 |
|---------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <code>PORT</code>                                                                           | Puerto HTTP (usar <b>3001</b> para alinear con el front) |
| <code>GEOTRENDS_DOCUMENTS_ROOT</code>                                                       | Carpeta de docs ( <code>data</code> relativo al backend) |
| <code>GEMINI_API_KEY</code> / <code>GEMINI_MODEL</code>                                     | Gemini                                                   |
| <code>GROQ_API_KEY</code> / <code>GROQ_MODEL</code>                                         | Groq                                                     |
| <code>GROQ_MAX_DOCUMENT_CHARS</code>                                                        | Opcional: techo de chars del doc a Groq                  |
| <code>DEEPSEEK_API_KEY</code> / <code>DEEPSEEK_MODEL</code> / <code>DEEPSEEK_API_URL</code> | DeepSeek                                                 |
| <code>OPENAI_API_KEY</code> / <code>OPENAI_MODEL</code>                                     | OpenAI                                                   |
| <code>HUGGINGFACE_API_KEY</code> / <code>HUGGINGFACE_MODEL</code>                           | Hugging Face                                             |
| <code>OLLAMA_BASE_URL</code> / <code>OLLAMA_MODEL</code>                                    | Ollama local                                             |

Para el **Agente inteligente** mínimo operativo:

- 1. `GEMINI_API_KEY` (recomendado siempre: default + fallback)
- 2. `GROQ_API_KEY` (preguntas rápidas)
- 3. `DEEPSEEK_API_KEY` (analítico; si falta, las complejas van a Gemini)

Las keys **nunca** van al frontend: solo viven en `backend/.env` .

---

## 9. Variables de entorno (frontend)

---

Archivo: `frontend/.env.local`

**Estado en el repo:** este archivo **sí está versionado en Git**. Quien clone ya lo recibe. No necesitas crearlo a menos que quieras cambiar la URL del backend.

```
NEXT_PUBLIC_AGENT_API_URL=http://localhost:3001/api/chat
NEXT_PUBLIC_COURSE_ID=geotrends
NEXT_PUBLIC_LESSON_ID=
```

| Variable                  | Efecto                                                    |
|---------------------------|-----------------------------------------------------------|
| NEXT_PUBLIC_AGENT_API_URL | Base del chat; de ahí se derivan las otras URLs           |
| NEXT_PUBLIC_COURSE_ID     | Se manda en cada POST/GET como <code>courseId</code>      |
| NEXT_PUBLIC_LESSON_ID     | Documento inicial opcional; vacío → sentinel hasta elegir |

## 10. Datos / bibliotecas

Los documentos viven bajo `backend/data/` (configurable). Estructura de ejemplo:

```
data/
├─ Ficha de Servicios/    ← Biblioteca en la UI
├─ Guia generica/         ← Biblioteca
└─ Manuales/              ← Biblioteca
```

- Añadir una biblioteca nueva = crear una carpeta de primer nivel dentro del root.
- Añadir documentos = copiar PDF/DOCX/XLSX/MD/TXT dentro (recursivo).
- Reiniciar el backend no es estrictamente necesario para archivos nuevos (se listan al vuelo); el cache se invalida por `mtime` al cambiar un archivo.

Archivos `.doc` antiguos (no `.docx`) **no** se indexan.

## 11. Historial de conversación

| Dónde                               | Comportamiento                                                                    |
|-------------------------------------|-----------------------------------------------------------------------------------|
| Frontend Smart                      | Acumula mensajes en estado; envía <code>history</code> sin el welcome             |
| Frontend paneles fijos              | UI de chat, pero el POST normalmente manda solo el mensaje actual                 |
| Backend <code>chatHistory.ts</code> | <code>normalizeHistory</code> + <code>trimHistory</code> (máx <b>12</b> mensajes) |
| Gemini / Groq / DeepSeek            | Reciben historial                                                                 |
| OpenAI / HF / Ollama                | <b>No</b> reciben historial hoy                                                   |

## 12. Decisiones de diseño (por qué está así)

1. **Varios paneles + Smart:** servir para demos/comparación de proveedores, y un modo producto unificado.
2. **Routing heurístico (no ML):** reglas simples, predecibles, sin costo extra; fácil de ajustar en `providerRouter.ts`.
3. **Truncado agresivo en Groq:** el proveedor rechaza payloads grandes (413); es mejor recortar que fallar.

- 4. **Sin base de datos:** el corpus es la carpeta; el chat no se persiste (reiniciar página / cambiar doc borra el hilo).
- 5. **Sin auth:** MVP local. Antes de exponer a internet hay que añadir autenticación y rate limit.
- 6. **Sin embeddings:** MVP rápido; mejorar RAG sería el siguiente salto de calidad en respuestas multi-doc.
- 7. **Integración futura Open edX / Vtiger:** la idea era cambiar solo `context.ts` para traer contexto por API; el front casi no cambia.

### 13. Limitaciones conocidas (importante para el sucesor)

- 1. Sin autenticación ni rate limiting.
- 2. Búsqueda solo lexical (no vectorial / embeddings).
- 3. `courseId` se valida pero no particiona el corpus en disco.
- 4. Duplicación fuerte entre los 6 paneles del frontend (candidatos a un `BaseAgentPanel` ).
- 5. Sin streaming → UX de espera en preguntas largas.
- 6. Cache de documentos en memoria sin tope.
- 7. Sin tests automatizados.
- 8. Errores HTTP 500 genéricos al cliente.
- 9. Carpeta `frontend/agent/` vacía.
- 10. Sin deploy productizado (Docker/CI): solo `npm run dev` / `build + start` locales.

### 14. Entrega operativa (lo que suele olvidarse al salir de la empresa)

Esta sección es crítica: el código solo no basta para que el proyecto arranque.

#### 14.1 Secretos y archivos que NO vienen en el clone

| Ítem                              | ¿Está en git?                  | Qué hacer en la entrega                                                                                      |
|-----------------------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>backend/.env</code>         | No ( <code>.gitignore</code> ) | Pasarlo por canal seguro (OneDrive privado, chat interno). El sucesor lo pone en <code>backend/.env</code> . |
| <code>backend/data/</code>        | No (gitignore)                 | Subir ZIP a OneDrive. El sucesor lo descomprime en <code>backend/data/</code> (ver sección 2).               |
| <code>frontend/.env.local</code>  | Sí (trackeado)                 | <b>No hace falta entregarlo aparte.</b> Ya viene en el clone con la URL local.                               |
| <code>backend/.env.example</code> | Sí                             | Plantilla sin secretos.                                                                                      |

#### Cómo entregar `data/` (quien sale de la empresa)

```
cd backend
zip -r geotrends-documentos-data.zip data
# Subir geotrends-documentos-data.zip a OneDrive y compartir el link
```

En el mensaje de traspaso escribe algo como:

1. Clona el repo.

2. Pon el `.env` que te pasé en `backend/.env`.

3. Descarga el ZIP de OneDrive y déjalo como `backend/data/` (con las carpetas Biblioteca adentro).

4. Sigue la sección 2 del PDF de documentación.

5. `npm install` + `npm run dev` en `backend/` y en `frontend/`.

14.2 Checklist de traspaso humano

- ☐

Transferir ownership / acceso al repo GitHub `isabelasring/agente`
- ☐

Entregar `backend/.env` actual (o regenerar keys y actualizar `.env`)
- ☐

Entregar o ubicar la carpeta `backend/data/` con PDFs/docs
- ☐

Indicar quién paga / dueño de cuentas Gemini, Groq, DeepSeek (billing)
- ☐

Probar juntos un chat Smart de punta a punta en la máquina del sucesor
- ☐

(Opcional) Rotar keys después del handoff si la saliente las tenía personales

14.3 Problemas frecuentes (troubleshooting)

| Síntoma                                       | Causa habitual                                            | Qué revisar                                                                                                        |
|-----------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Front: "No pude conectar con el backend..."   | Backend apagado o puerto distinto                         | <code>npm run dev</code> en backend;<br><code>PORT=3001</code> ; URL en <code>.env.local</code>                    |
| Bibliotecas vacías en el wizard               | Falta <code>backend/data/</code> o root mal configurado   | <code>GEOTRENDS_DOCUMENTS_ROOT=data</code><br>y que existan subcarpetas                                            |
| "Falta configurar GEMINI_API_KEY..."          | Key ausente o vacía                                       | <code>backend/.env</code>                                                                                          |
| Respuestas de Groq fallan / fallback a Gemini | Payload grande (413) o key inválida                       | Logs <code>[smart]</code> / <code>[chat]</code> ;<br><code>GROQ_API_KEY</code> ; shrink en <code>askGroq.ts</code> |
| Preguntas complejas siempre a Gemini          | No hay DeepSeek                                           | <code>DEEPSEEK_API_KEY</code> en <code>.env</code>                                                                 |
| Ollama no responde                            | Servicio local no corre                                   | <code>ollama serve</code> + modelo<br><code>OLLAMA_MODEL</code>                                                    |
| Preview PDF en blanco                         | Ruta inválida o archivo no indexable                      | Path relativo correcto; extensión <code>.pdf</code>                                                                |
| Chat "se borra" al cambiar documento          | Intencional ( <code>key</code> en <code>page.tsx</code> ) | No hay persistencia de historial                                                                                   |

14.4 Regenerar el PDF de esta documentación

Si actualizas el `.md` y quieres el PDF otra vez:

```
# Requiere pandoc + Google Chrome instalados (macOS)
pandoc DOCUMENTACION-HANDOFF.md -t html5 -o /tmp/handoff-geobot.html --standalone \
--metadata title="Geobot – Documentación de entrega"
# Abrir el HTML en Chrome → Imprimir → Guardar como PDF
# o usar Chrome headless --print-to-pdf=DOCUMENTACION-HANDOFF.pdf
```

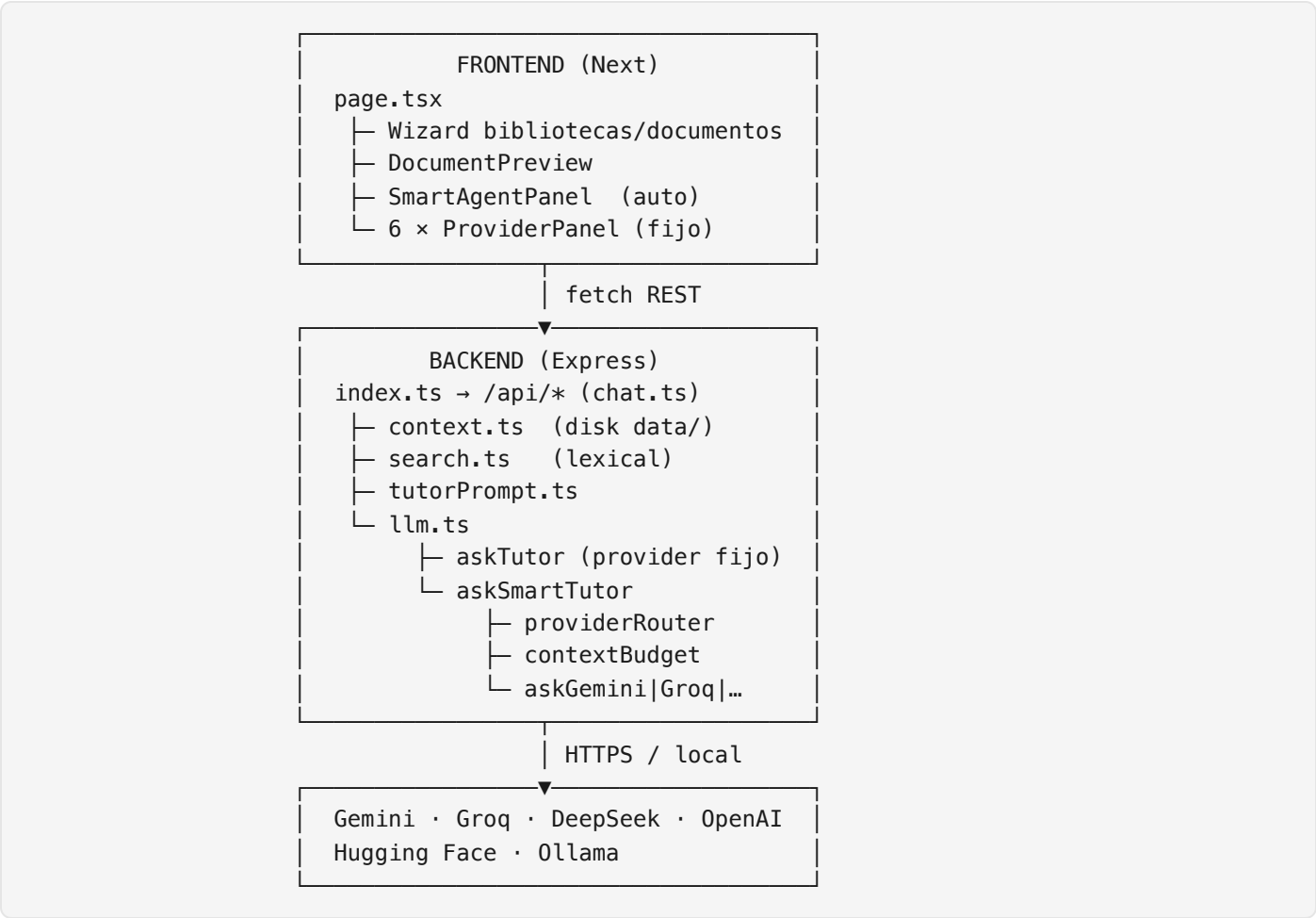
## 15. Checklist rápido para el nuevo responsable

- ☐ Acceso al repo + backend/.env + carpeta data/
- ☐ Clonar, npm install en backend/ y frontend/
- ☐ Confirmar PORT=3001 y GEOTRENDS\_DOCUMENTS\_ROOT=data en backend/.env
- ☐ Confirmar frontend/.env.local → http://localhost:3001/api/chat
- ☐ npm run dev en ambos; abrir http://localhost:3000
- ☐ Ver bibliotecas en el wizard (si no, falta data/ )
- ☐ Pregunta corta → debería ir a **Groq** (badge + routingReason)
- ☐ Pregunta analítica ("compara...", "explica en detalle...") → **DeepSeek** o Gemini
- ☐ Leer: providerRouter.ts , llm.ts , routes/chat.ts , SmartAgentPanel.tsx

## 16. Mapa "si quiero cambiar X, voy a Y"

| Quiero...                                 | Archivo(s)                                                                                              |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Cambiar reglas del auto-route             | backend/src/services/providerRouter.ts                                                                  |
| Cambiar qué es "pregunta compleja"        | backend/src/services/contextBudget.ts                                                                   |
| Cambiar el tono / instrucciones del tutor | backend/src/prompts/tutorPrompt.ts                                                                      |
| Añadir un LLM nuevo                       | Adapter en backend/src/<nuevo>/ , registrar en llm.ts + panel en frontend/<nuevo>/ + opción en page.tsx |
| Meter un LLM al Smart                     | Ampliar RoutedProvider + pickProvider + presupuesto en contextBudget                                    |
| Cambiar endpoints o contrato              | backend/src/routes/chat.ts + frontend/shared/agentConfig.ts                                             |
| Cambiar URLs del API en el front          | frontend/.env.local / agentConfig.ts                                                                    |
| Cambiar carpetas de documentos            | GEOTRENDS_DOCUMENTS_ROOT + contenido de backend/data/                                                   |
| Mejorar preview PDF/DOCX/Excel            | frontend/shared/DocumentPreview.tsx + rutas document en chat.ts                                         |
| Unificar paneles duplicados               | Extraer componente base desde *Panel.tsx                                                                |
| Persistir chats / usuarios                | Hoy no existe: habría que diseñar DB + auth                                                             |

# 17. Diagrama mental del sistema



Última actualización del handoff: julio 2026. Si el código y este documento divergen, prioriza el código en `providerRouter.ts`, `llm.ts`, `routes/chat.ts` y `SmartAgentPanel.tsx`.